

Mullvad WireGuard Egress — Status Summary

Revision: 1 **Last modified:** 2026-07-02T20:20:00Z **Status:**

Companion summary of [Status.md](#) (§11.4.56 two-audience).

2026-07-02: the persistent one-device Mullvad WireGuard egress feature is **PROVEN** — a persistent single-device Mullvad identity (device "**groovy rabbit**", id df4184d4-b6e9-4c2e-b7f7-16e15d4e55a2) is registered + stored (§11.4.10, gitignored .env), **live egress** is proven (mullvad_exit_ip=true, deterministic 3/3), and the **kill-switch / fail-closed** guarantee is proven (tunnel-DOWN ⇒ all egress **BLOCKED**, deterministic 3/3, §11.4.68). Two items remain honest: the full proxy-data-plane-through-gluetun end-to-end path is **PENDING**, and live DNS is **OPERATOR-BLOCKED** (operator-deferred 2026-07-02).

Page 1 — For the operator / stakeholders (plain language)

We proved that the proxy can send traffic out through a **Mullvad VPN tunnel** using a permanent, single-device VPN identity on the operator's account — and, just as importantly, that if the tunnel ever drops, **nothing leaks to the open internet**. Every "proven" below is backed by a saved proof file captured while the tunnel was actually running, so none of these results can be faked. The VPN credentials are kept private (in an ignored config file, never in the code).

What we proved this session:

- **A permanent, single-device VPN identity.** We registered one dedicated Mullvad device — named "groovy rabbit" — so the proxy always presents as the *same* device on the VPN. The private credentials are stored only in a private, git-ignored config file.
- **Traffic really exits through the VPN.** Mullvad's own "am I Mullvad?" check confirmed `mullvad_exit_ip = true` — the traffic left through the VPN, not the open internet. We confirmed this **three times in a row**. The exit location moved between runs (Prague, then Dusseldorf, then San Jose) because the VPN rotates its relays, but every single time the traffic exited through Mullvad — the guarantee that matters held every run.

- **The "kill-switch" works (the safety-critical one).** We deliberately forced the VPN tunnel **down** and confirmed that *all* internet access was immediately **blocked** — again, three times in a row. We also saw the VPN's own internal health-check get blocked by the same firewall while the tunnel was down, which independently confirms the block is real. This is the guarantee that the proxy never sends your traffic out unprotected if the VPN fails.

What is not done yet (stated honestly):

- **Full end-to-end proxy-through-VPN wiring:** the VPN tunnel and its kill-switch are proven, but the final engineering step — routing everyday proxy traffic all the way through this tunnel end-to-end — is still to be done and captured.
- **Live DNS** is on hold at the operator's request (deferred on 2026-07-02, part of the Let's Encrypt certificate setup) — this is an operator decision, not a VPN problem.

Bottom line: the Mullvad VPN egress identity, its live traffic-exit proof, and its kill-switch safety are all proven with saved evidence. One engineering step (full proxy-through-VPN wiring) and one operator-deferred item (live DNS) remain, and both are stated openly. Nothing is overstated.

Page 2 — For software engineers

§11.4.45 captured-evidence reconciliation for the Mullvad WireGuard egress feature. Verdicts cite the literal invariants from each qa-results/ artefact. Secrets never appear — only the device name/id + evidence paths (§11.4.10).

Aspect	Status	Evidence
Persistent one-device WireGuard identity	PASS	qa-results/verification/mullvad_egress_20260702T161312Z/PROOF.txt — device "groovy rabbit", id df4184d4-b6e9-4c2e-b7f7-16e15d4e55a2, registered via the Mullvad app API; config stored in gitignored .env (§11.4.10)
Live Mullvad egress	PASS	qa-results/verification/mullvad_egress_20260702T161312Z/PROOF.txt (+ egress.json, egress_iter2.json, egress_iter3.json) — mullvad_exit_ip=true DETERMINISTIC 3/3 (§11.4.50); exits Prague / Dusseldorf / San Jose (relay rotation), invariant mullvad_exit_ip=true holds every run;

Aspect	Status	Evidence
Kill-switch / fail-closed	PASS (§11.4.68)	host routing untouched (§11.4.174), containers torn down (§11.4.14) qa-results/mullvad_killswitch/20260702T165417Z/08_killswitch_loop.txt (+ SUMMARY.md) — gluetun FIREWALL=on; atomic tun0-DOWN (confirmed before+after) ⇒ raw-IP egress RC=4 (firewall DROP), DETERMINISTIC 3/3 (§11.4.50); the ~8s-post-drop success was the AUTO-RESTORED tunnel, not a leak (07_drop_blocked.txt)
Kill-switch firewall mechanism	PASS	qa-results/mullvad_killswitch/20260702T165417Z/04_iptables_up.txt — iptables OUTPUT policy DROP + only -A OUTPUT -o tun0 -j ACCEPT ⇒ fail-closed by construction
gluetun healthcheck corroboration	PASS	qa-results/mullvad_killswitch/20260702T165417Z/09_gluetun_firewall_block_evidence.txt — gluetun's own healthcheck ⇒ write: operation not permitted while tun0 down (out-of-band firewall-block corroboration, §11.4.13-class)
Full proxy-data-plane-through-gluetun e2e	PENDING	Not yet captured — the proven tunnel + kill-switch, wired through the full dynamic proxy stack data-plane, is the next step (§11.4.6 — no evidence cited)
Live DNS (LE Phase 4/6)	OPERATOR-BLOCKED	Operator-deferred DNS 2026-07-02 (Let's Encrypt Phase 4 of 6) — downstream operator decision, not an egress-feature defect

Composes §11.4.45 (integration-status doc), §11.4.56 (two-audience summary), §11.4.5/§11.4.69 (captured evidence), §11.4.68 (fail-closed / kill-switch positive evidence), §11.4.50 (deterministic 3/3), §11.4.10 (no secrets), §11.4.6 (no-guessing — PENDING/OPERATOR-BLOCKED stated, not hidden), §11.4.14 (container teardown), §11.4.174 (host routing untouched).